CS 111

Foundations of Program Design

Week 2 Lecture

Variable assignment & scope. Boolean operators.

No class on Monday

Labor Day holiday

No Monday lab next week.

Quiz 1

Handed out during lecture.

Quiz 1

5 minutes remaining.

This lecture

1. int, boolean, String types
2. Variable assignment
3. Variable scope
4. (break)
5. Handy terminal shortcuts
6. Common git commands: `clone`, `fetch/pull`, `status`, `add/rm`, `commit/push`
7. Lab: Push your first commit to Course-Information-Fall2026.

No class on Monday!

Logistics

Quizzes will be handed back at the end of class **next Wednesday**.

If you have any questions about assignments, learning resources, or the course contents, ask your instructors at office hours or by email.

- Paria's office hours are now Thursdays, 11:30am - 1:30pm.

  On Harvey Science 4th floor, in the CS tutoring lounge.
- Paria also holds tutoring hours from 1:30pm - 2:30pm Thursdays, and 10am - 12noon Fridays.

–

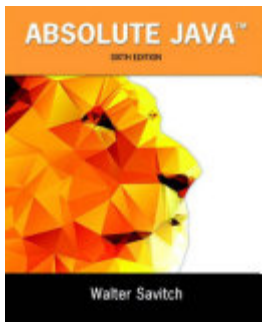
- **No office hours on Monday** for the holiday. Wednesday office hours as usual, 11am - 12noon.
- I will host remote office hours 1 - 3pm on Tuesday.

  Sign up for a 15min slot during break or after class.

Course resource update

I realize the Head First Java 3rd edition text is poorly designed as a reference work.

Dr. Kristin Jones has a copy of Absolute Java 6th Edition by Walter Savitch, which we can borrow during office hours.



Please come by my office hours to discuss anything related to language reference.

Literal values

A **literal** is a fixed value that you have written in your Java code. We'll cover three **types** of literal today.

- **int**eger values: Numbers without a decimal place, including negative numbers (e.g., `42`, `-7`, `0`).
- **boolean** values: Either `true` or `false`. Often used to control program flow, such as branching (`if/else`) or looping (`while` or `do/while` or `for`).
- **String** values: Text enclosed in straight double quotes (e.g., `"Hello, Java"`, `"123"`).
  To include a double-quote, escape it with a backslash (e.g. `"I said \"Hi,\" to my friend."`).

Expression values

In Java, an **expression** is a combination of variables, operators, literals, and method calls that evaluates to a single value.

The **expression value** is the final result produced after Java carries out the required calculations or operations.

Every expression produces a value: Evaluating an expression computes a result that can be assigned to a variable, passed as an argument, or used in a condition.

Common Examples of Expression Values

Expression Type	Example Expression	Evaluated Value	Data Type
Arithmetic	`10 + 5 * 2`	`20`	`int`
Comparison	`15 > 20`	`false`	`boolean`
String Concatenation	`"Java " + 25`	`"Java 25"`	`String`
Compound Expression	`(5 + 2) == 7`	`true`	`boolean`
Method Call	`"hello".toUpperCase()`	`"HELLO"`	`String`

How Java Uses Expression Values

```java
public class Comparison {
    public static void main(String[] args) {
        // These are variable assignments - we'll addresss them in a moment.
        int a = 5;
        int b = 10;

        System.out.println("Total: " + (a + b)); // Total: 15
        System.out.println("Result: " + (a == b)); // Result: false
    }
}
```

= assigment vs == equality

Variable assignment in Java stores a value into a declared memory location using the single equals sign (`=`). The value on the right-hand side is evaluated first and then stored in the variable on the left-hand side.

```
int score = 100; // Declaration and initialization
score = 150;     // Reassigns 'score' to a new value
```

Mixing up variable assigment with equality tests is one of the most common beginner traps in Java.

Assignment Operator (`=`)	Equality Test (`==`)
Action: Sets a value in a variable.	**Question:** Asks whether two values are equal.
Returns the assigned value.	Returns a `boolean` (`true` or `false`).
`int x = 5;`	`if (x == 5) { ... }`

Exercises

- **Types Must Match:** The variable type on the left must be compatible with the evaluated expression on the right (e.g., you cannot assign a `String` to an `int`).

```java
public class AssignmentDemo {
    public static void main(String[] args) {
        boolean isReady = false;

        System.out.println("Initial value of isReady: " + isReady);
        System.out.println();

        if (isReady == true) {
            System.out.println("I can't be reached");
        } else {
            System.out.println("I'm working as expected");
        }
        System.out.println("Final value of isReady: " + isReady);
    }
}
```

- **Caution**: Assigment and equality operators look similar, but behave differently. `boolean isReady; if (isReady = true) { ... }` will silently reassign `isReady` to `true`, and then run the branch.